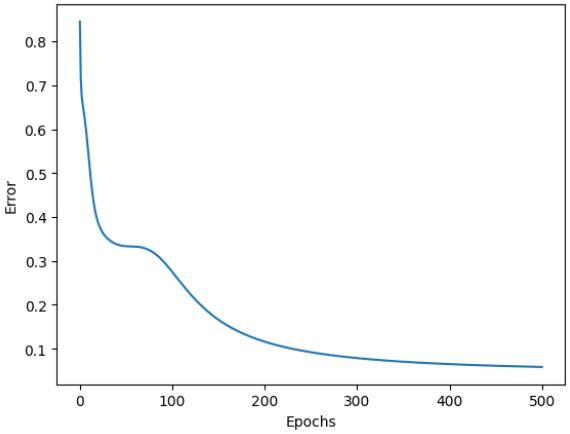
Inteligencia artificial

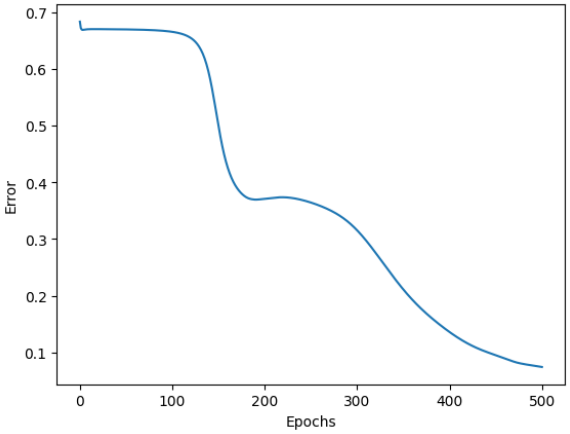
Gruintal Polanco, Guillermo Cristian

Más capas en el perceptrón multicapa (Iris)

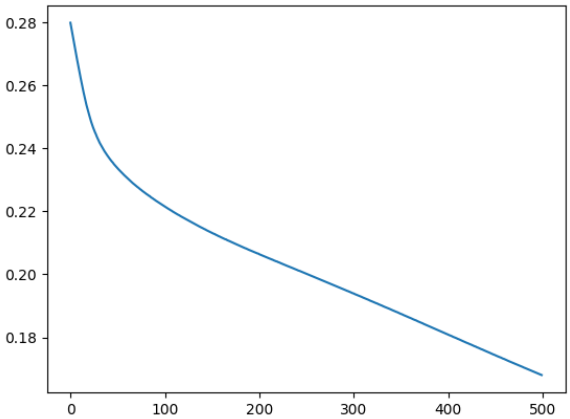
Capturas

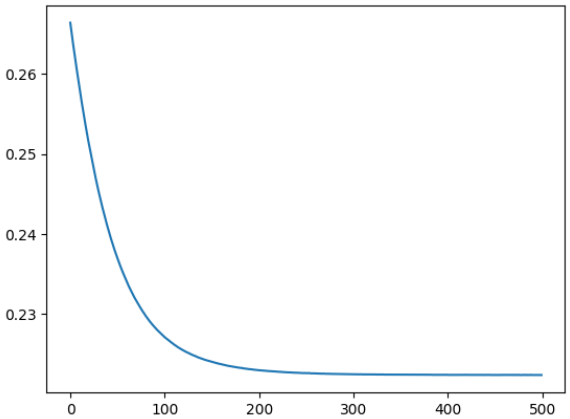
Curvas de error



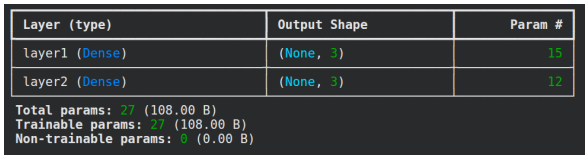


Keras



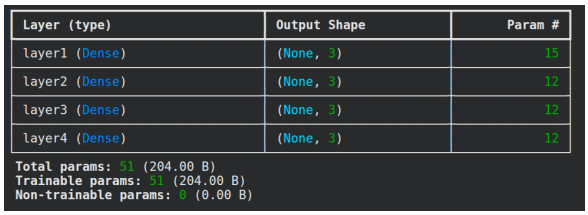


Summary



Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)



Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 51 (204.00 B)
Trainable params: 51 (204.00 B)
Non-trainable params: 0 (0.00 B)

2. Reporte de resultados

¿Bajar más el error al añadir dos capas, o se estancó / empeoró? ¿Igual en NumPy y en Keras?

Al añadir dos capas lo que pasó es que empeoró y se estancó, en NumPy el error aumentó considerablemente cuando se agregaron las capas, mientras que en Keras, al agregar las capas, el error descendió muy despacio. En ambos casos el desempeño fue peor al agregar capas.

¿Las curvas de la notebook 01 y de Keras se parecen con la misma topología? Si no, ¿qué diferencias de implementación podrían explicarlo (orden de los datos, inicialización, vectorización, etc.)?

Las gráficas de ambas curvas son decrecientes, pero presentan diferencias en la escala y el ritmo de descenso, ya que en NumPy se sumaron errores cuadráticos de las 3 salidas, mientras que Keras usó MeanSquaredError(), lo que hace que arranque en una escala más baja. De igual manera, NumPy usó una distribución uniforme simple, mientras que Keras emplea Glorot Uniform, lo que evita que se sature al inicio.

Con sigmoides apiladas y MSE, ¿tiene sentido que una red más profunda no aprenda mejor en Iris? Relaciónalo con lo que viste en las gráficas.

Sí tiene sentido, ya que la función sigmoide tiene una derivada pequeña, por lo que al hacer la retropropagación el gradiente se vuelve infinitesimal en las primeras capas, lo que frena el aprendizaje temprano.

4. Evidencias de Ejecución

